Лабораторная работа №1

Предобработка данных. Знакомство с pandas

Перед тем как обучать модель машинного обучения, необходимо подготовить данные. В реальных задачах датасеты часто содержат пропущенные значения, выбросы и категориальные признаки, которые алгоритмы не могут обработать напрямую. Неподготовленные данные могут привести к ошибочным выводам и низкой точности модели.

В данной лабораторной работе разобраны основные этапы предобработки данных:

1. **Заполнение пропущенных значений**
2. **Нормализация данных**
3. **Преобразование категориальных данных**

Язык программирования Python

Самым популярным языком программирования, используемым в анализе данных и машинном обучении, является Python.

Python — мультипарадигмальный высокоуровневый язык программирования общего назначения с динамической строгой типизацией и автоматическим управлением памятью, ориентированный на повышение производительности разработчика, читаемости кода и его качества, а также на обеспечение переносимости написанных на нём программ. Язык является ***полностью объектно-ориентированным*** в том плане, что ***всё является объектами***. Необычной особенностью языка является выделение блоков кода отступами. Сам язык известен как интерпретируемый и используется в том числе для написания скриптов. Недостатками языка являются зачастую более низкая скорость работы и более высокое потребление памяти написанными на нём программами по сравнению с аналогичным кодом, написанным на компилируемых языках, таких как C или C++.

Установка Python на компьютер

Скачать установщик Python можно с официального сайта: Welcome to Python.org. Рекомендуется использовать версию python 3.10, так как версия 3.8 уже закончила свой жизненный цикл, а версия 3.9 в скором времени также перестанет получать обновления. Инструкцию по установке можно найти здесь.

Среда разработки

Существует множество сред разработки для языка Python, но среди них особенно выделяются две — PyCharm и JupyterLab.

PyCharm — десктопная среда разработки от компании JetBrains, ориентированная на работу с Python. Доступена в двух версиях: ***PyCharm Community Edition*** — бесплатная версия, находится под лицензией Apache License, и ***PyCharm Professional Edition*** — расширенная версия продукта, обладающая дополнительной функциональностью, является проприетарным ПО. На данный момент компания JetBrains закрыла возможность бесплатного получения студенческой лицензии *PyCharm Professional Edition* для обучающихся из белоруских вузов. Бесплатную *PyCharm Community Edition* можно скачать здесь. Данная IDE поддерживает плагины от сторонних разработчиков, которые могут частично дать функционал доступный в платной версии.

JupyterLab — это интерактивная среда разработки для работы с блокнотами, кодом и данными, предназначенная для работы с широким набором языков программирования, в частности с Python. Блокноты (***Jupyter Notebook***), в данном случае, — это крайне удобный инструмент для создания красивых аналитических отчетов, так как он позволяет хранить вместе код, изображения, комментарии, формулы и графики. Подробнее об этой IDE можно узнать в этих статьях: “Что есть в новом JupyterLab для пользователей?” и “Особенности Jupyter Notebook, о которых вы (может быть) не слышали” .

Формат блокнотов позволяет размещать блоки кода в ячейках (***cells***). Ячейки можно запускать отдельно друг от друга, а также неоднократно одну и ту же ячейку. Данные между вызовами ячеек хранятся в ядре блокнота, что позволяет не перезапускать весь код в случае ошибки при его написании и вести работу поэтапно, проверяя работоспособность каждого блока кода. Этому также способствует вывод результатов работы (не только числа и текст, но также картинки и графики, а при установке некоторых плагинов даже аудио и видео) под ячейкой после успешного выполнения блока кода в ней. Ядро блокнота можно очистить перезапустив его.

Файлы блокнотов для языка Python хранятся в формате .ipynb. Работу с данным форматом также поддерживают платная версия PyCharm и Visual Studio Code, с установленным плагином для работы с блокнотами. При записи на диск, файл также сохраняет результаты вывода (текст, графики и т.д.), а не только код. Поэтому, как было сказано выше, данный формат позволяет эффективно выполнять задачи анализа данных и предоставления

отчетов, так как не требует повторного запуска кода для демонстрации результатов.

Устанавливается JupyterLab довольно просто, с использованием пакетного менеджера **pip**, который устанавливается на ПК вместе с Python. Для загрузки IDE в командной строке требуется выполнить следующую команду:

```
pip install jupyterlab
```

После успешной загрузки JupyterLab можно запустить. Делается это также через командную строку. JupyterLab использует веб-интерфейс, то есть попросту говоря работа идет в браузере. Через командную строку же локально запускается веб-сервер IDE по адресу http://localhost:8888. Команда для запуска (для работы с файлами из определенной директории, выполняйте команду заранее перейдя в нужную папку в командной строке):

```
jupyter lab
```

Помимо всего, JupyterLab поддерживает расширения (аналог плагинов в других IDE). В JupyterLab уже предустановлены некоторые расширения для визуализации данных, работы с файлами разных форматов и т.д.. Однако расширение для автозаполнения и подсказок в коде отсутствует в стандартном наборе.

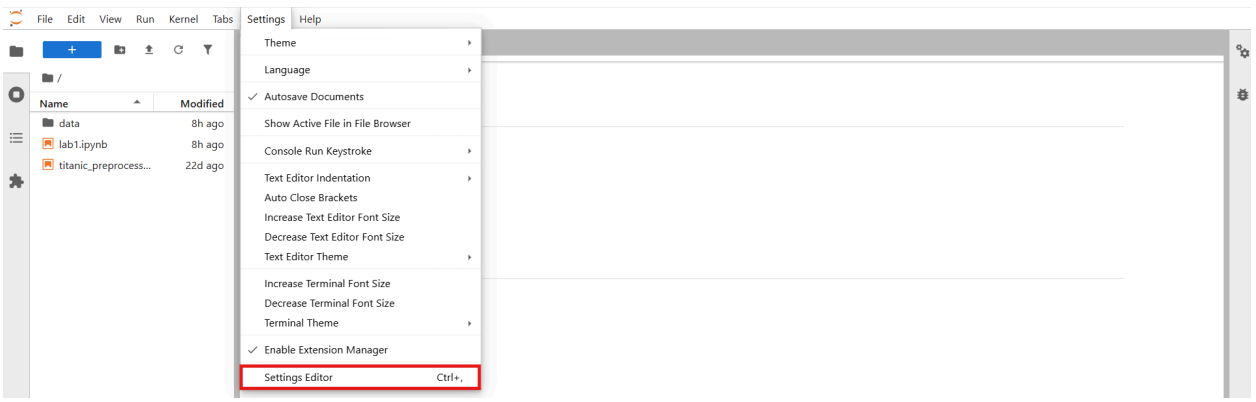
Данный недостаток можно устранить установив расширение jupyterlab-lsp. Также как и для JupyterLab, установка производится через командную строку:

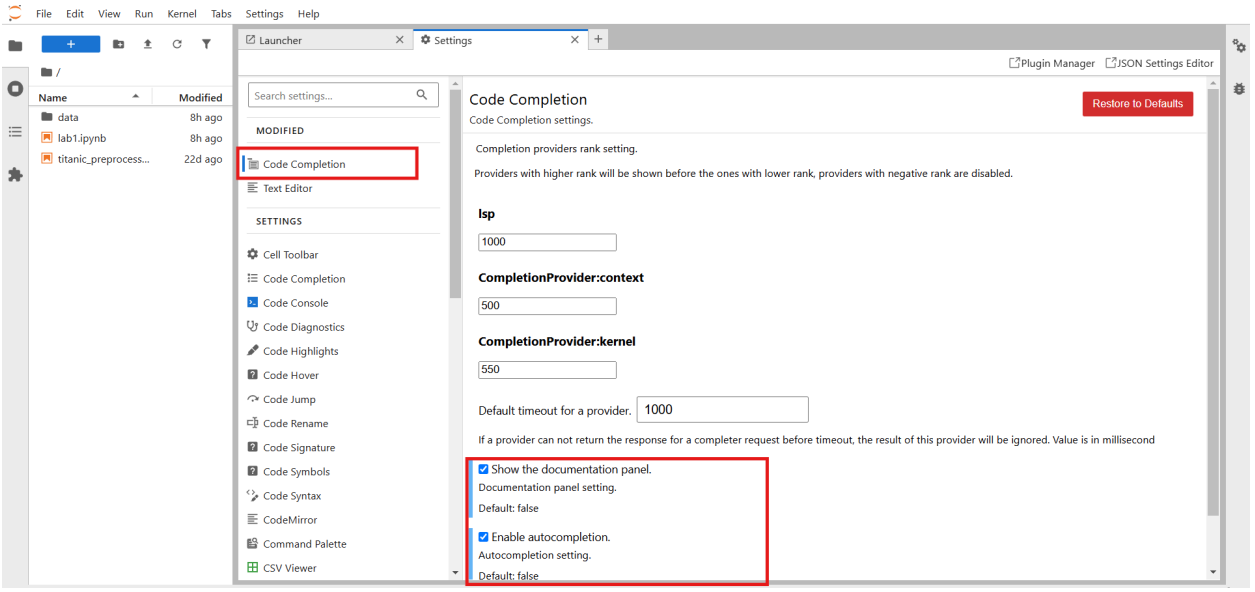
```
pip install jupyterlab-lsp
```

Загрузка поддержки для языка Python:

```
pip install python-lsp-server[all]
```

Далее, если ваш JupyterLab сервер был запущен, перезапустите его. После перезапуска, в расширенных настройках JupyterLab требуется активировать автозаполнение:





После этого при написании кода в ячейка блокнота появятся варианты автозаполнения, а также информация из документации. О всех полезных особенностях этого расширения можно узнать на GitHub странице проекта.

Установка пакетов (библиотек)

По умолчанию в Python в качестве менеджера пакетов используется pip. Чтобы установить библиотеку с помощью pip в командной строке нужно выполнить следующую команду:

```
pip install <package_name>
```

где <package_name> — имя требуемой библиотеки. Например, для загрузки библиотеки pandas используйте следующую команду:

```
pip install pandas
```

Эта команда установит последнюю версию библиотеки. Но если нужна какая-то конкретная версия, отличная от последней, ее можно указать после двойного равно:

```
pip install pandas==2.2.0
```

Если скачиваемый пакет имеет зависимости от других пакетов, они автоматически “подтянутся” вместе с установкой первого. Хотя в редких случаях это приходится делать вручную. Для просмотра всех установленных пакетов используйте:

```
pip list
```

Так же в реальных проектах на Python обычно создается файл со всеми зависимостями используемыми в данном проекте. Типичное название такого файла — requirements.txt. Он содержит список всех требуемых для запуска проекта библиотек с указанием их версий. Чтобы установить все библиотеки из данного файла, используете команду:

```
pip install -r requirements.txt
```

Создать свой файл с зависимостями проекта можно следующей командой:

```
pip freeze > requirements.txt
```

Эта команда создаст список из всех библиотек, которые установлены для используемого интерпретатора.

По умолчанию все пакеты скачиваются в основной интерпретатор, находящийся в директории с Python. Однако различные проекты требуют не только разного набора библиотек для запуска, но и использование различных версий одних и тех же библиотек. Это означает, что при чередовании работы между двумя и более проектами, вам придется каждый раз загружать нужные версии библиотек для проекта. Проект может требовать использования как более новых версий пакетов (с новым функционалом), так и более старых версий (где нужный функционал из библиотеки еще не удален). Также сами библиотеки могут перестать корректно работать, если пакеты, от которых первые зависят, будут несовместимы по версиям.

Для решения данной проблемы для каждого проекта создается свое **виртуальное окружение** перед началом работы над ним.

Виртуальное окружение Python

Виртуальное окружение позволяет изолировать зависимости разных проектов, избегая конфликтов между библиотеками. Оно изолирует пакеты, требуемые для конкретного проекта, от основного интерпретатора. Для создания своего виртуального окружения выполните следующие действия:

1. В командной строке (или через проводник) создайте папку, в которой будет лежать проект:
   ```
   mkdir my_ml_project
   cd my_ml_project
   ```
2. В командной строке введите команду для создания виртуального окружения (вместо **<your_directory_name>** введите своё название,

обычно используется venv):

```
python -m venv <your_directory_name>
```

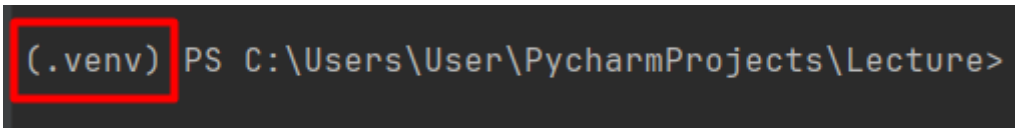
3. Активируйте виртуальное окружение:
    a. Windows:

```
<your_directory_name>\Scripts\activate
```

    b. Linux/MacOS:

```
source <your_directory_name>/bin/activate
```

После выполнения последнего пункта командная строка примет следующий вид — в начале командной строки появится надпись формата "*(.<your_directory_name>)*", в данном случае "(.venv)":



Теперь все ваши пакеты будут загружаться в папку виртуального окружения, при условии что вы работаете в командной строке с запущенным виртуальным окружением.

Также, чтобы ваша IDE использовала зависимости из виртуального окружения требуется:

1. Для JupyterLab — запустить веб-сервер JupyterLab из активированного виртуального окружения.
2. Для PyCharm — при создании проекта в PyCharm, виртуальное окружение автоматически создается в той же папке. Для использования данного виртуального окружения либо активируйте его через командную строку, либо используете встроенный терминал в PyCharm, где виртуальное окружение уже запущено.

Библиотека pandas

Pandas — это библиотека с открытым исходным кодом, которая позволяет легко и интуитивно работать с реляционными или маркированными данными. Она предоставляет различные структуры данных и операции для работы с числовыми данными и временными рядами. Она предлагает инструмент для очистки и обработки данных. Это самая популярная библиотека Python для анализа данных.

Ядром pandas являются две структуры данных, в которых происходят все операции:

- **Series** — представляет собой одномерный маркированный массив, способный хранить любой тип данных (целые числа, строки, числа с плавающей точкой, объекты Python и т. д.)
- **DataFrame** — это двумерная маркированная структура данных со столбцами потенциально разных типов. Вы можете думать о ней как о электронной таблице или таблице SQL, или **словаре объектов Series**. Это, как правило, наиболее часто используемый объект pandas.

Больше информации о данной библиотеке и том как ею пользоваться вы можете узнать на официальном сайте проекта и на сайте GeegForGeeks. Здесь же будут рассмотрены основные инструменты pandas для предобработки данных.

Базовые операции Python

Для использования в коде программ библиотек, их требуется импортировать с помощью ключевого слова *import*:

```
import pandas as pd
```

Как было сказано выше, в Python **все является объектами**, поэтому взаимодействие с библиотекой происходит через импортированный из неё объект. В данном случае мы импортировали объект ***pandas*** и присвоили ему псевдоним ***pd*** (*as pd*).

Также полезными базовыми функция Python будут:

- Вывод строкового представления объекта консоль:

```
print(obj)
```

В Jupyter блокнотах для вывода представления объекта под ячейкой достаточно просто указать этот объект в последней строчке ячейки. Причем тот же JupyterLab уже имеет предустановленные расширения для красивого и интерактивного вывода информации из объектов популярных типов.

- Получение типа объекта:

```
type(obj)
```

Загрузка данных

Датасет для предобработки можно найти на сайте Kaggle. На нем представлен широкий выбор датасетов. В их описании вы можете узнать в чем состоит задача модели, которую вы будете обучать на этих данных, какие данные содержатся в датасете, поучаствовать в соревновании на самую точную модель с другими участниками, посмотреть решения от других пользователей и многое другое.

Обычно данные представлены и хранятся в файле формата csv (Comma-Separated Values). Чтобы загрузить данные из файла в объект класса DataFrame используйте следующий метод pandas:

```
df = pd.read_csv("titanic.csv")   # путь к файлу
```

Пройдемся по основным методам для просмотра и получения информации о датасете при работе с объектом класса DataFrame.

Получение первых пяти строк датасета:

```
df.head()
```

Получение первых n строк датасета:

```
df.head(n)
```

С помощью следующего метода можно получить информацию о датасете, начиная с названий столбцов и типов данных в них, заканчивая количеством непустых значений в каждом столбце:

```
df.info()
```

Ниже представленная строчка вернет объект класса Series, содержащий информацию о названии столбцов и типе данных хранимых в них:

```
df.dtypes
```

Для получения названий колонок используйте

```
cols = df.columns
```

в данном случае вернется объект класса Index представляющий собой **неизменяемую** последовательность. Для получения названий колонок в виде обычного списка python примените также метод *tolist()*:

```
df.columns.tolist()
```

Для проверки наличия (отсутствия) некого объекта obj в последовательности Index примените *in (not in)*:

```
obj = 'Name'
if obj in cols:
    print('Dataset has column Name')
obj = 'Drowned'
if obj not in cols:
    print('Dataset hasn’t column Drowned')
```

Метод *drop()* объекта класса Index вернет новую последовательность без объектов, переданных в качестве параметра:

```
cols_without_age = train.columns.drop('Age')
cols_without_age_and_name = train.columns.drop(['Age',
'Name'])
```

При использовании объекта DataFrame в цикле foreach, в ходе итераций перебираются все названия столбцов в строковом виде:

```
for col in df:
    print(col) #вывод всех названий столбцов
```

Для получения объекта столбца используются квадратные скобки с указанием имени столбца (также можно указать числовой индекс):

```
age_column = df['Age']
```

В этом случае вернется объект класса Series хранящий имя и все значения столбца. Таким же образом можно получить и новый DataFrame состоящий из выбранных столбцов:

```
age_and_name_df = df[['Age','Name']]
```

Метод класса DataFrame *select_dtypes()* позволит получить новый DataFrame, содержащий только столбцы определенный типов. Для указания фильтра по типу используются два параметра метода *include* и *exclude* для включения и исключения типов столбцов соответственно в возвращаемый DataFrame:

```
numeric_df = df.select_dtypes(include='number')
```

```
not_numeric_df = df.select_dtypes(exclude='number')
```

В качестве значений параметров *include* и *exclude* могут быть использованы строки:

- *'number'* — соответствует числовым типам;
- *'object'* — чаще всего применяется для фильтрации строковых типов, однако если в датасете присутствуют колонки типа object, но не представляющие собой набор строк, они также будут возвращены;
- и другие.

Создать новые или перезаписать существующие колонки в датафрейме можно следующим образом:

```
#добавление нового столбца VIP
df['VIP'] = df1['VIP_Status']
#добавление двух новых столбцов Deck и Side
df[['Deck', 'Side']] = df2[['CabinDeck', 'CabinSide']]
```

Удалить столбец позволяет метод *drop()*:

```
df_without_name = df.drop('Name', axis='columns')
df.drop('Name', axis='columns', inplace= True)
```

Первый параметр указывает на имя удаляемого из датафрейма объекта. *axis* говорит о том по какой оси искать нужный объект, значение может быть как индекс оси, так и название, что более наглядно и не дает запутаться. Последний параметр, *inplace,* отвечает за то, будет ли полученный в ходе выполнения метода датафрейм записан в переменную, для которой вызван метод, или нет. По умолчанию *inplace = False*, поэтому метод вернет преобразованный датафрейм в качестве выходного значения; при *inplace=True — None*.

Чаще всего данные представлены в виде одного большого файла, содержащего все данные. Однако для предотвращения переобучения модели датасет разбивают на обучающую и тестовую выборки. В Python для этой задачи можно использовать инструменты из библиотеки для машинного обучения scikit-learn. Более глубокое ознакомление с её возможностями будет в следующих лабораторных работах. На данном этапе нам понадобится функция разбиения датасета:

```
from sklearn.model_selection import train_test_split
```

```
train_df, test_df = train_test_split(df,
test_size=0.3, random_state=42)
```

У функции два выходных значения — обучающая и тестовая выборки, полученные из поданного в качестве первого параметра датасета. Параметр *test_size* определяет долю данных, которые попадут в тестовую выборку; *random_state* используется в качестве начального числа для генератора псевдослучайных чисел на основе которого данные будут перемешаны, это также способствует избежанию переобучения модели.

Заполнение пропущенных значений

Пропущенные значения (NaN) могут возникать по разным причинам: ошибки при сборе данных, невозможность получения информации и т. д. При помощи метода *isnull()* можно получить матрицу значений bool, содержащую True в позициях, где в вызывающем метод датафрейме имеется пропущенное значение:

```
nan_matrix = df.isnull()
```

Метод *sum()* считает сумму числовых значений для каждого столбца и возвращает объект Series, содержащий имя столбца и сумму значений в нем. Причем для значений типа bool — True считается в сумму как единица, а False как нуль:

```
nan_matrix.sum()
```

Пропущенные значения могут негативно повлиять на результат обучения модели, поэтому нужно либо удалить строки содержащие такие значения, либо заменить их неким другим значением:

- **Мода (самое частое значение)** – подходит для категориальных признаков:

```
cabin_mode = df['Cabin'].mode()[0]
```

- **Медиана (среднее значение после сортировки)** – устойчива к выбросам:

```
age_median = df['Age'].median()
```

- **Среднее значение** – наиболее часто используемый метод для числовых данных:

```
weight_mean = df['Weight'].mean()
```

Для заполнения пропущенных значений используется метод *fillna()*:

```
df['Cabin'].fillna(cabin_mode, inplace=True)

df['Age'].fillna(age_median, inplace=True)

df['Weight'].fillna(weight_mean, inplace=True)
```

Параметр *inplace* используется в таком же контексте, как и в методе *drop()* описанном выше.

Нормализация данных

Нормализация данных — это процесс преобразования числовых переменных в общую шкалу, не искажая различия в диапазонах значений. Этот подход улучшает работу алгоритмов, чувствительных к диапазону значений (например, градиентный спуск).

Часто используемые методы нормализации:

1. **Минимально-максимальное масштабирование (MinMaxScaler)**
   - Преобразует значения в диапазон от 0 до 1.
   - Формула:

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

   - Используется, когда важны относительные различия между значениями.
2. **Z-оценка (StandardScaler)**
   - Преобразует данные так, чтобы среднее равнялось 0, а стандартное отклонение было 1.
   - Формула:

$$X' = \frac{X - \mu}{\sigma}$$

Для применения нормализации к численным столбцам понадобится функционал библиотеки scikit-learn:

```
from sklearn.preprocessing import MinMaxScaler,
StandardScaler

scaler = MinMaxScaler()  # или StandardScaler()
```

Полученный таким образом объект *scaler* ещё не до конца подготовлен к применению. Внутри данный объект содержит модель для получения нормализованных значений. При создании *scaler*, для модели ещё не известны параметры в формуле нормализации. Попытка получить значения в таком случае спровоцирует ошибку.

Для того чтобы настроить модель, нужно передать объекту *scaler* данные для вычисления параметров формулы. Делается это через метод *fit()*:

```
scaler.fit(df['Age'])
```

После этого можно получить нормализованные значения:

```
df['Age'] = scaler.transform(df['Age'])
```

Также существует метод для одновременного вычисления параметров модели и получения нормализованных значений:

```
df['Age'] = scaler.fit_transform(df['Age'])
```

Обратите внимание, что повторное применение для одного и того же *scaler* методов *fit()* и *fit_transform()* с разными входными данными изменит параметры модели вычисляющей нормализованные значения!

Преобразование категориальных данных в численные

Алгоритмы машинного обучения не могут работать с текстовыми данными, поэтому перед обучением их кодируют. Есть два способа кодировки: с помощью последовательных чисел и с помощью one-hot encoding.

При кодировании последовательностью чисел, каждому уникальному значению категориального признака определяется свое порядковое натуральное число.

One-hot encoding (OHE) — это метод преобразования категориальных переменных в числовой формат. Он создает новый бинарный столбец для каждого уникального значения категории в исходном датасете.

Принцип работы:

1. Для каждого уникального значения категориальной переменной создается новый столбец.
2. В новых столбцах ставится 1, если наблюдение имеет соответствующую категорию, и 0 в противном случае.

За OHE-преобразование в библиотеке pandas отвечает метод *get_dummies()*:

```
df = pd.get_dummies(df, columns=['Destination'], drop_first=True)
```

Входными параметрами выступают: датафрейм, используемый для преобразования; *columns* — список столбцов, к которым применяется OHE; *drop_first* — флаг, отвечающий за то будет ли отсутствовать столбец для одного из значений категории (если в изначальном датасете наблюдение имело данное значение, в новом представлении этому будет соответствовать значения 0 во всех остальных столбцах для данной категории).

Выходным же значением метода является новый датафрейм, полученный после OHE-преобразования поданного на вход датафрейма. При этом столбцы не участвующие в OHE-преобразовании останутся неизменными, а также изначальный датафрейм не претерпит никаких изменений.

Сохранение обработанных данных и выгрузка на GitHub

Для сохранения предобработанных данных в CSV-файл можно используют следующий метод:

```
df.to_csv("processed_titanic.csv", index=False)
```

Для выгрузки кода и результатов на GitHub следуйте следующим пунктам:

1. Создайте репозиторий на GitHub.
2. Перейдите в терминал и выполните:
   git init

```
git add .

git commit -m "Added lab1 code"

git branch -M main

git remote add origin https://github.com/ваш-аккаунт/ваш-репозиторий.git

git push -u origin main
```

3. Отправьте ссылку на репозиторий преподавателю.

Практическое задание

1. Загрузить данные с Kaggle – выбрать датасет (например, Титаник)
2. Вывести с помощью python данные из датасета на экран
3. Получить количество пропущенных значений для каждого столбца в датасетах
4. Заполнить пропущенные значения в датасете модой/медианой/средним значением и показать, что они действительно были заполнены
5. Провести нормализацию данных
6. Провести преобразование категориальных данных так, чтобы в будущем это не привело к переобучению модели машинного обучения и датасет имел удобоваримый вид
7. Выгрузить код и результаты на гитхаб (сбросить ссылку преподавателю)